Table of Contents

Creating the App 1
React Native Router Introduction 1
Tip: When Emulator Not Refreshing 4
Pressable Components 5
Navigation with Tabs 7
Icons 9
Managing Input with State 11
Using the Debugger 13
Radio Button List 14
List / Detail View 20

Creating the App

Example 1: Create the app

To create the app, make a folder where you have full permissions. Then open the folder in VS Code. Launch an Android emulator from Android Studio and wait for it to launch so the home screen appears. Next, open the VS Code terminal by navigating to **Terminal | New Terminal**. Then cd into the folder and run this command to create the app:

```
npx create-expo-app@latest --template blank

(prefix this with sudo on a Mac)
```

I experienced several permissions issues on my Mac. In terminal I had to grant read write permission on my project folder and sub folders:

```
sudo chown -R $USER ./
```

To start the app, in the same folder as package.json run:

```
npx expo start
```

Be sure to press **a** to run the app in the Android emulator which should already be running.

React Native Router Introduction

The style of expo routing with index.jsx placed inside an app folder as the app entry point will be used for all React Native examples which are covered from this point onwards in this course.

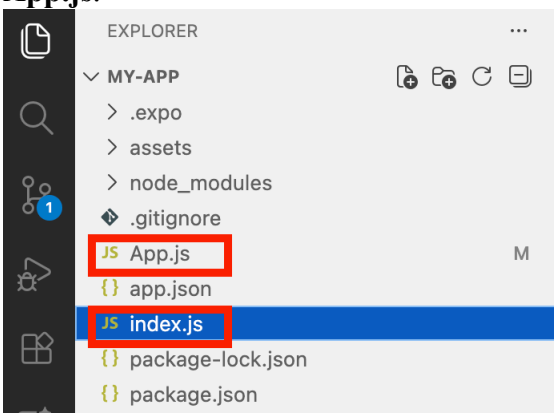
Example 2: Customizing the App

This example will make adjustments to use the React Native router. To enable routing install the following dependencies. The most current collection of required dependencies are documented at https://docs.expo.dev/router/installation/#manual-installation

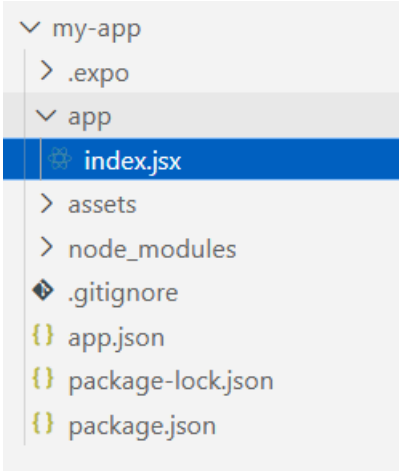
Run this command in the VS Code terminal to get the dependences:

```
npx expo install expo-router react-native-safe-area-context react-native-screens expo-linking
expo-constants expo-status-bar
```

Then, in your project folder delete the app entry point which is defined **index.js**. Also delete **App.js**.



Next create an app folder and add a new **index.jsx** file in the app folder. Be sure to use lower case.

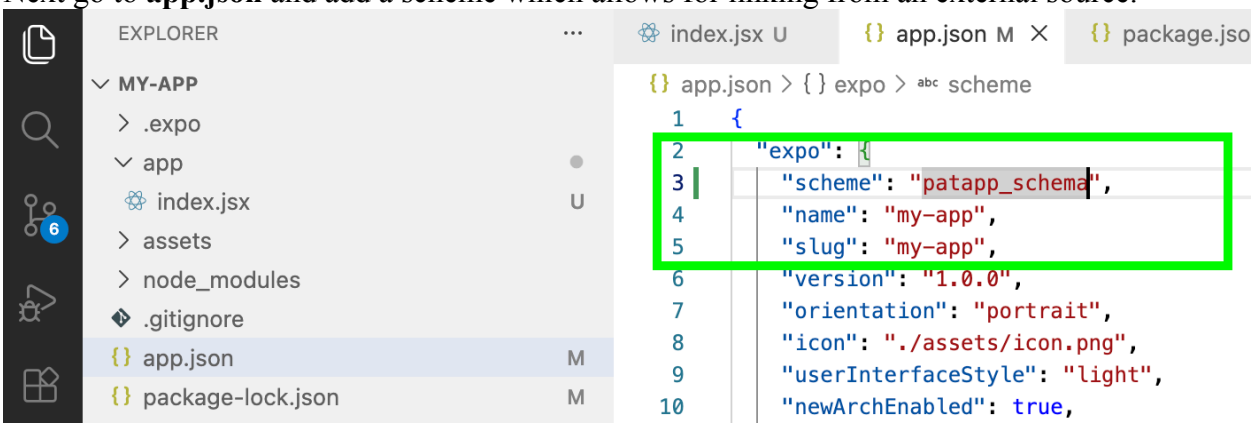


The code inside **index.jsx** defines a component called Home. The Home component is actually made up of other components including View and Text. The tags to implement these components are not actually HTML tags – they are React Native components which are designed to run in iOS and Android environments. The native components must be imported so they can be used.

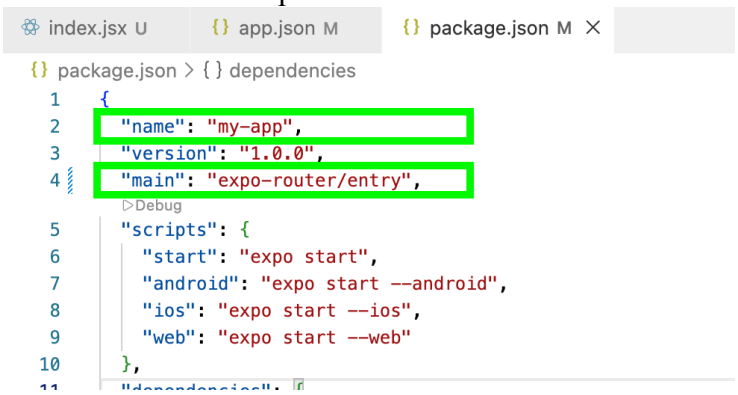
app\index.jsx

```
import { StyleSheet, Text, View } from 'react-native'
import React from 'react'

const Home = () => {
  return (
    <View>
      <Text>This is my home screen</Text>
    </View>
  )
}
export default Home
const styles = StyleSheet.create({})
```

Next go to **app.json** and add a scheme which allows for linking from an external source.



In **package.json**, ensure the name matches the name in the app.json file. Change the *main* attribute to use the expo-router.



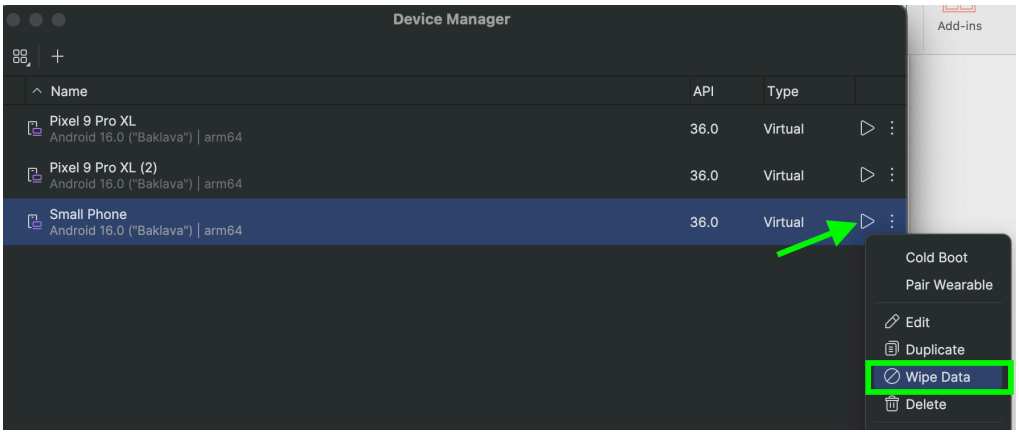
When running the app, the output appears but it is formatted badly. The unformatted text will appear at the top left and may barely be visible due to the lack of formatting.

Tip: You may need to press r to refresh the app when running it to have the new routing take effect.

Tip: When Emulator Not Refreshing

When working on React Native apps, you may find that the app occasionally does not refresh when pressing **r** in the VS Code terminal. There may be better methods to handle this situation but currently these steps work well for me:

1. Stop the app from running.
2. Shut the emulator down.
3. Launch the Virtual Device Manager from Android Studio, wipe the emulator you want to use and then restart the emulator.

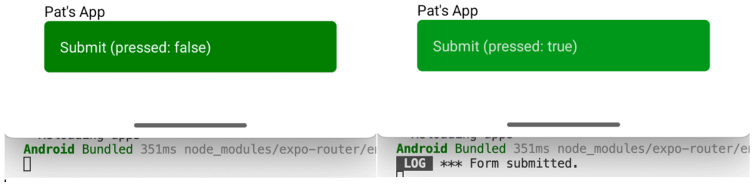


4. After the emulator has finished launching and the home screen is visible, relaunch React Native app.
5. If the emulator still does not to refresh, try using simple code that has been proven to work such as the code for Example 2.a

Pressable Components

Example 3: Pressable Components

This example demonstrates how a pressable can be applied to change the color and log output during the press event. For this example, a pressable is applied to a Text component.



To build this example, start with Example 2. Then replace the contents of **index.jsx** with the following:

app\index.jsx

```
import { StyleSheet, Text, View, Pressable, Colors } from 'react-native'

const Home = () => {
  const handleSubmit = () => {
    console.log("*** Form submitted.")
  }

  return (
    <View style={{ flex: 1, justifyContent: 'center', padding: 40 }}>
      <View style={styles.container}>
        <Text style={styles.title}>Pat's App</Text>

        <Pressable
          onPress={handleSubmit}
          style={({ pressed }) => [styles.btn, pressed && styles.pressed]}
        >
          {({ pressed }) => (
            <Text style={{ color: '#f2f2f2' }}>
              Submit (pressed: {pressed ? "true" : "false"})
            </Text>
          )}
        </Pressable>
      </View>
    </View>
  )
}

export default Home
const styles = StyleSheet.create({
  btn: {
    backgroundColor: 'green',
    padding: 15,
    borderRadius: 5,
  },
  pressed: {
    opacity: 0.8,
  }
})
```

Exercise 1 (1 mark)

What does the Pressable component do, and how is it different from a regular Button?

Exercise 2 (1 mark)

How is the handleSubmit function called in Example 3?

Exercise 3 (1 mark)

In your own words, explain how the styling for the button is modified during a press event.

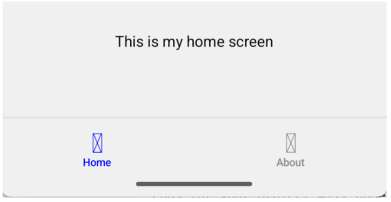
Exercise 4 (1 mark)

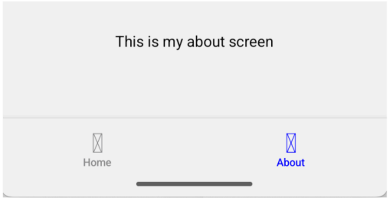
What is the purpose of ({ pressed }) => (...) inside the JSX?

Navigation with Tabs

Example 4: Building an App with Tabs

This example builds a tabs app to navigate to home and about pages.





The tabs are defined in the _layout.jsx file.

app_layout.jsx

```
import { Tabs } from 'expo-router';

export default function RootLayout() {
  return (
    <Tabs
      screenOptions={{
        headerShown: false,
        tabBarActiveTintColor: 'blue',
        tabBarStyle: { backgroundColor: '#f2f2f2' }}}>
      <Tabs.Screen name="index" options={{ title: 'Home'}} />
      <Tabs.Screen name="about" options={{ title: 'About'}}/>
    </Tabs>
  );
}
```

The Home screen is defined in index.jsx.

app\index.jsx

```
// app/index.jsx
import { StyleSheet, Text, View } from 'react-native';

export default function Home() {
  return (
    <View style={styles.container}>
      <Text>This is my home screen</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, alignItems: 'center', justifyContent: 'center' },
});
```

Similarly, the About screen is defined in about.jsx.

app\about.jsx

```
import { StyleSheet, Text, View } from 'react-native';

export default function About() {
  return (
    <View style={styles.container}>
      <Text>This is my about screen</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, alignItems: 'center', justifyContent: 'center' },
});
```

Exercise 5 (1 mark)

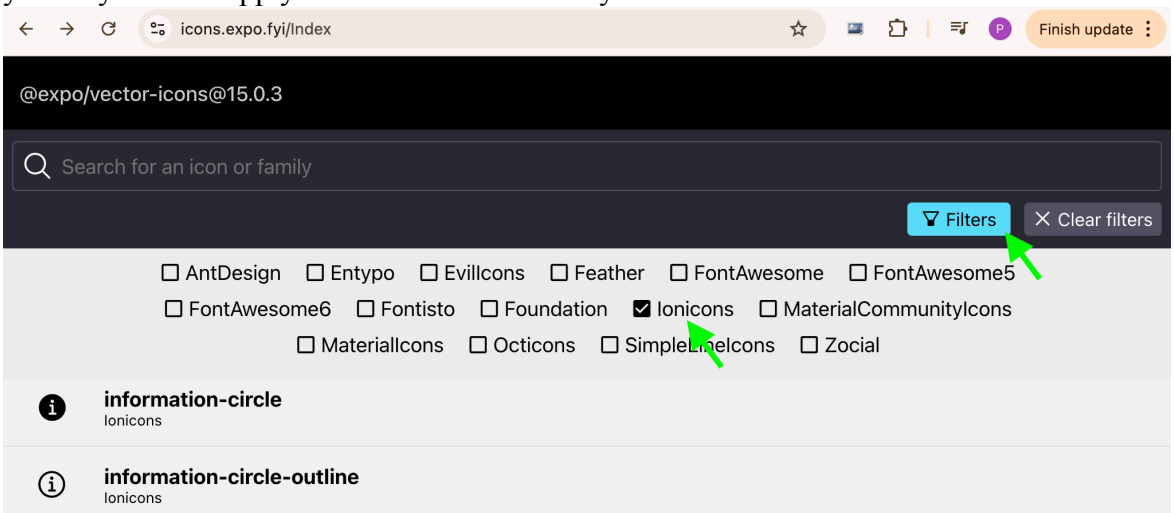
Explain how the tab changes color when it is active.

Exercise 6 (1 mark)

Explain how the screen is selected with the Tab.Screen from Expo router.

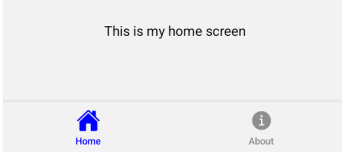
Icons

The icon reference can be found at https://icons.expo.fyi/Index. Note that this material will be using ionicons from @expo-vector/icons in this document. As a result, when searching for icons you may want to apply a filter for ionicons only.



Example 5: Adding icons

This example continues with the example from Example 3 to add icons to the tabs.



Be sure these dependencies exist in **dependencies** section of **package.json**. I had to add the react-dom dependency so I suspect you may need to do this as well.

```
"react-dom": "19.2.0",
```

Then, prepare for a clean build with the new dependencies by running the following two commands:

```
rm -rf node_modules package-lock.json
npm install
```

The new code needed to show the icons for the tabs is highlighted in yellow.

app_layout.jsx

```jsx
// app/_layout.jsx
import React from 'react';
import { Tabs } from 'expo-router';
import { Ionicons } from '@expo/vector-icons';

export default function RootLayout() {
  return (
    <Tabs
      screenOptions={{
        headerShown: false,
        tabBarActiveTintColor: 'blue',
        tabBarStyle: { backgroundColor: '#f2f2f2' },
      }}
    >
      <Tabs.Screen
        name="index"
        options={{
          title: 'Home',
          tabBarIcon: ({ color, size }) => (
            <Ionicons name="home" color={color} size={size} />
          ),
        }}
      />

      <Tabs.Screen
        name="about"
        options={{
          title: 'About',
          tabBarIcon: ({ color, size }) => (
            <Ionicons name="information-circle" color={color} size={size} />
          ),
        }}
      />
    </Tabs>
  );
}
```

Exercise 7 (4 marks)

Create a third screen with your name on it that is accessed with a tab that has your first name on it. Then launch the app and click on the new tab. Take a screenshot of the app showing the new screen and your new tab with your names on them.

Managing Input with State

In React Native, state variables are created with the useState hook using the syntax:

```
const [inputVal, setInput] = useState("");
```

The declaration contains these three features:

inputVal is the current state value
setInput is the function to change its value
useState("") sets the initial value (as an empty string)

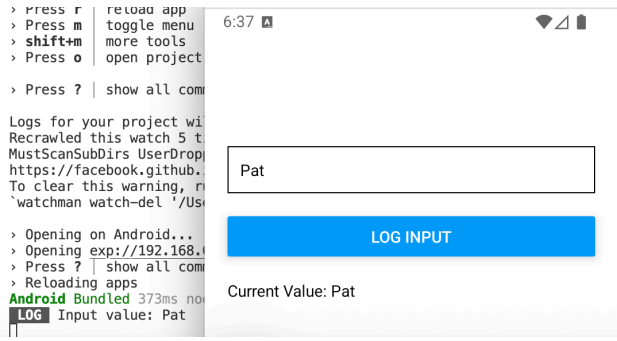
You update the value by calling the setter function:

```
setInput("new text");
```

React then re-renders the component so the UI shows the updated state.

Example 6: Managing Input with State

This example shows how state can be used to store content from a **TextInput** component and update all UI components that reference it.



Starting with Example 5, replace index.jsx with this version.

app\index.jsx

```
import { View, Text, TextInput, Button } from 'react-native';
import { useState } from 'react';

export default function App() {
  const [inputVal, setInput] = useState("");

  // separate function to log input
  const logInput = () => {
    console.log("Input value:", inputVal);
  };

  return (
    <View style={{ padding: 20, marginTop: 100 }}>
      <TextInput
        placeholder="Type something..."
        value={inputVal}
        onChangeText={setInput}
        style={{ borderWidth: 1, padding: 10, marginBottom: 20 }}
      />

      <Button
        title="Log Input"
        onPress={logInput}
      />

      <Text style={{ marginTop: 20 }}>
        Current Value: {inputVal}
      </Text>
    </View>
  );
}
```

Exercise 8 (1 mark)

Which line updates the value stored in inputVal?

Exercise 9 (1 mark)

What does the value property of the TextInput do?

Exercise 10 (1 mark)

What does onChangeText={setInput} mean? Why can we pass setInput directly?

Exercise 11 (3 marks)

Modify Example 6 so the text field is cleared after the value is logged:

```
setInput("");
```

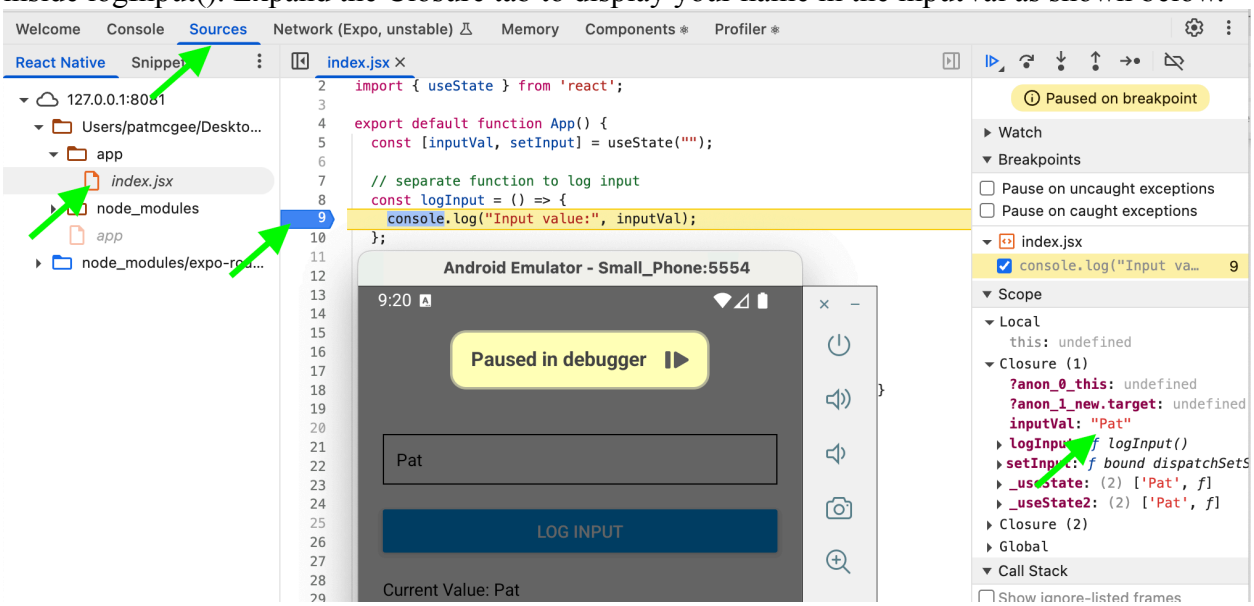
Test the app to make sure the new behaviour works correctly. Then show the revised logInput() function.

Using the Debugger

The debugger can be opened by pressing **j** in the VS Code terminal while the app is running in the emulator.

Exercise 12 (3 marks)

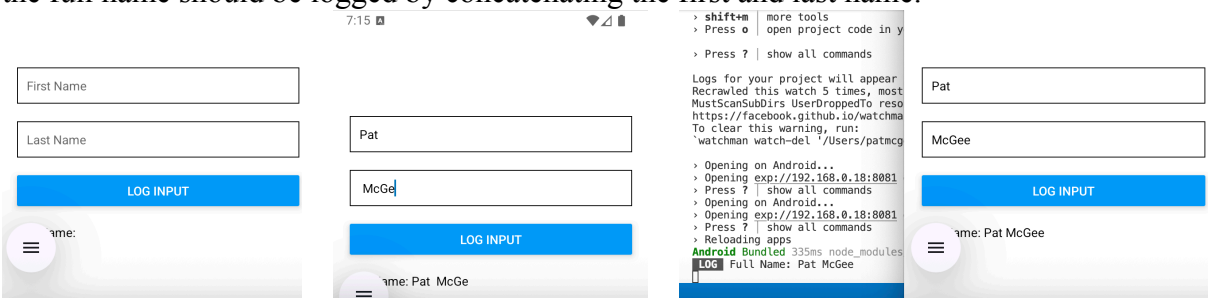
While Example 6 is running, open the debugger by pressing **j** in the terminal. Then choose the Sources tab and select the index.jsx file. Set a breakpoint inside the logInput() function. Then enter your name in the input and submit it. Adjust the panel so the app is halted at this breakpoint inside logInput(). Expand the Closure tab to display your name in the inputVal as shown below.



Show a screenshot of your output:

Exercise 13 (3 marks)

Modify Example 6 to the first input stores and displays a first name. Add a second TextInput to store and display a second name with the same behaviour. When the user clicks the submit button the full name should be logged by concatenating the first and last name.



Show a screenshot of your output here with your first and last name in it:

Show the code for index.jsx here:

Radio Button List

Example 7: Radio Button List

This example shows how to implement a radio button list in React Native like the one shown below:

Pick a fruit:

- ○ Apple
- ○ Banana
- ◉ Cherry

Selected: Cherry

```
import { useState } from "react";
import { View, Text, Pressable } from "react-native";

export default function Home() {
  const [selected, setSelected] = useState(null);

  const options = ["Apple", "Banana", "Cherry"];

  return (
    <View style={{ flex: 1, justifyContent: "center", padding: 20 }}>
      <Text style={{ fontSize: 18, marginBottom: 20 }}>Pick a fruit:</Text>

      {options.map((option) => (
        <Pressable
          key={option}  // React requires unique key for each item
                        // in a list to update UI efficiently.
          onPress={() => setSelected(option)}
          style={{
            flexDirection: "row",
            alignItems: "center",
            marginBottom: 15,
          }}
        >
          {/* Outer Circle */}
          <View
            style={{
              height: 22,
              width: 22,
              borderRadius: 11,
              borderWidth: 2,
              borderColor: "#333",
              alignItems: "center",
              justifyContent: "center",
              marginRight: 10,
            }}
          >
            {/* Inner filled circle if selected */}
            {selected === option && (
              <View
                style={{
                  height: 12,
                  width: 12,
                  borderRadius: 6,
                  backgroundColor: "#333",
                }}
              />
            )}
          </View>
```

Exercise 14 (1 mark)

For Example 7, why is useState(null) setting null?

Exercise 15 (1 mark)

For Example 7, how does the app know which fruit is currently selected?

Exercise 16 (1 mark)

For Example 7, what happens when the user taps one of the Pressable items?

Exercise 17 (1 mark)

For Example 7, why do we need the key={option} inside the .map()?

Exercise 18 (1 mark)

For Example 7, explain what the code represented by `(...)` is doing in the expression
`{selected === option && (...)}`

Exercise 19 (1 mark)

How does React Native update the UI when *selected* changes?

Exercise 20 (2 marks)

Given these style properties for the Text component;

```
<Text style={{ color: "rgb(0, 200, 0)", fontWeight: "bold" }}>
```

Adjust Example 7 so the selected item is bold and purple.

Show a screenshot after making this change when an item is selected.

Show your modified code here:

Example 8: Encapsulating the Radio Button List

This example rearranges the radio list from Example 7 so it is contained in a re-useable component that is stored in a separate file. The component is contained a file called RadioList.jsx.

app\RadioList.jsx

```
import { View, Text, Pressable } from "react-native";

export default function RadioList({ options, selected, onChange }) {
  console.log("RadioList rendered with selected:", selected);
  return (
    <View>
      {options.map((option) => (
        <Pressable
          key={option}
          onPress={() => onChange(option)}
          style={{
            flexDirection: "row",
            alignItems: "center",
            marginBottom: 15,
          }}
        >
          {/* Outer Circle */}
          <View
            style={{
              height: 22,
              width: 22,
              borderRadius: 11,
              borderWidth: 2,
              borderColor: "#333",
              alignItems: "center",
              justifyContent: "center",
              marginRight: 10,
            }}
          >
            {/* Inner filled circle if selected */}
            {selected === option && (
              <View
                style={{
                  height: 12,
                  width: 12,
                  borderRadius: 6,
                  backgroundColor: "#333",
                }}
              />
            )}
          </View>

          <Text>{option}</Text>
        </Pressable>
      ))}
    </View>
  );
}
```

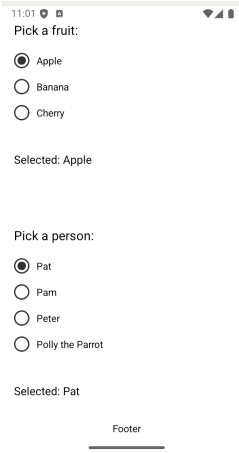
index.jsx can then import the component.

app/index.jsx

```
import { useState } from "react";
import { View, Text } from "react-native";
import RadioList from "./RadioList";   // adjust path as needed

export default function Home() {
  const [selected, setSelected] = useState(null);

  const options = ["Apple", "Banana", "Cherry"];

  return (
    <View style={{ flex: 1, justifyContent: "center", padding: 20 }}>
      <Text style={{ fontSize: 18, marginBottom: 20 }}>Pick a fruit:</Text>

      <RadioList
        options={options}
        selected={selected}
        onChange={setSelected}
      />
      <Text style={{ marginTop: 30, fontSize: 16 }}>
        Selected: {selected ? selected : "None"}
      </Text>
    </View>
  );
}
```

Exercise 21 (4 marks)

Modify Example 8 so it re-uses the radio button list to display names including yours. It must display the selected name at the bottom.



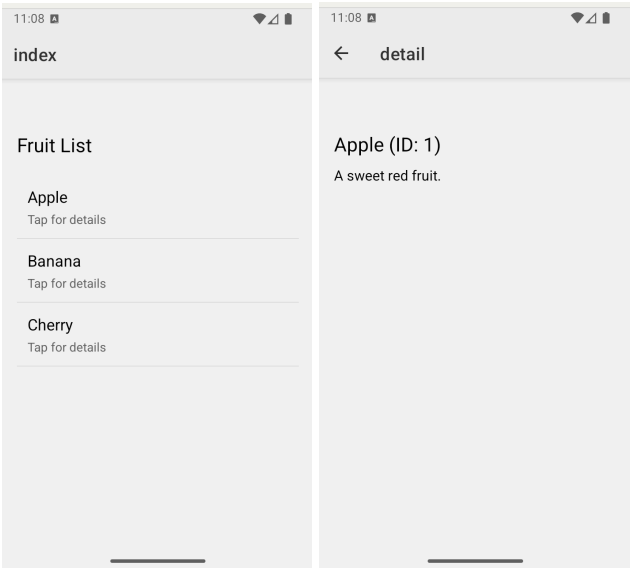
Show a screenshot of the screen after making this change.

Show your revised index.jsx file after making this change.

List / Detail View

Example 9: List Detail View

This example shows how to set up navigation to display a list which can be clicked to open a detail screen.



The layout page defines styling for the screen with the Stack component.

app/_layout.jsx

```
import { Stack } from "expo-router";

export default function RootLayout() {
  return (
    <Stack
      screenOptions={{
        headerStyle: { backgroundColor: "#eee" },
        headerTintColor: "#333",
      }}
    />
  );
}
```

The Detail screen retrieves parameters from the parent with the useLocalSearchParams() function.

app/detail.jsx

```
import { View, Text } from "react-native";
import { useLocalSearchParams } from "expo-router";

export default function Detail() {
  const { id, title, description } = useLocalSearchParams();

  return (
    <View style={{ flex: 1, padding: 20, paddingTop: 60 }}>
      <Text style={{ fontSize: 22, marginBottom: 10 }}>
        {title} (ID: {id})
      </Text>
      <Text style={{ fontSize: 16 }}>{description}</Text>
    </View>
  );
}
```

Here is the Home screen component which displays the objects with a FlastList that is scrollable and memory friendly.

app/index.jsx

```
import { View, Text, FlatList, Pressable } from "react-native";
import { Link } from "expo-router";

const DATA = [
  { id: "1", title: "Apple", description: "A sweet red fruit." },
  { id: "2", title: "Banana", description: "A yellow fruit rich in potassium." },
  { id: "3", title: "Cherry", description: "A small, juicy stone fruit." },
];

export default function Home() {
  return (
    <View style={{ flex: 1, padding: 20, paddingTop: 60 }}>
      <Text style={{ fontSize: 22, marginBottom: 20 }}>Fruit List</Text>

      <FlatList
        data={DATA}
        keyExtractor={(item) => item.id} // Use item id as unique key for each row.
        renderItem={({ item }) => (
          <Link
            href={{
              pathname: "/detail",
              params: { id: item.id, title: item.title,
                        description: item.description },
            }}
            asChild
          >
            <Pressable
              style={{
                padding: 12,
                borderBottomWidth: 1,
                borderColor: "#ddd",
              }}
            >
              <Text style={{ fontSize: 18 }}>{item.title}</Text>
              <Text style={{ color: "#666", marginTop: 4 }}>Tap for details</Text>
            </Pressable>
          </Link>
        )}
      />
    </View>
  );
}
```

Exercise 22 (1 mark)

How is the data passed to the flat list?

Exercise 23 (1 mark)

Why does each item in the FlatList need a keyExtractor?

Exercise 24 (1 mark)

How are title and description passed to the next screen? Explain what happens in index.jsx and detail.jsx to enable the parameter passing.

Exercise 25 (1 mark)

Modify the app so two items have the same id. Show the error that appears in the debugger (not the emulator).

Exercise 26 (4 marks)

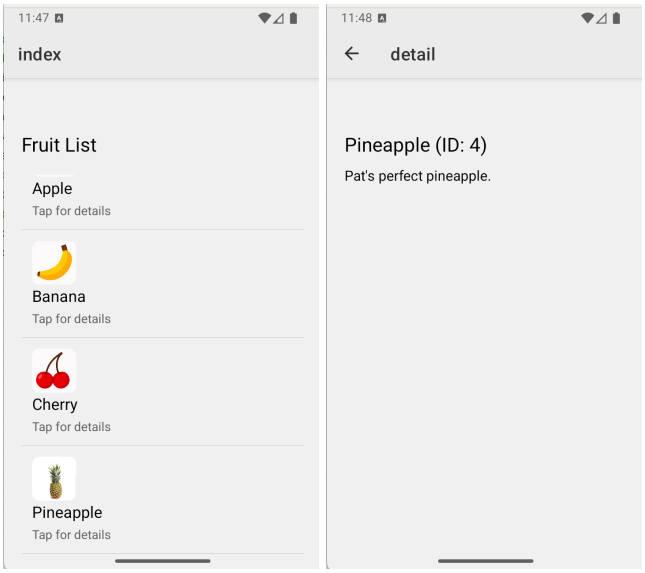
Images data can be defined in the list like the following:

```
{ id: "1", title: "Apple", description: "A sweet red fruit.", image: require("../assets/apple.png") }
```

Then the image can be displayed as part of a FlatList as shown here:

```
<Image
    source={item.image}
    style={{
    width: 50,
    height: 50,
    borderRadius: 8,
    }}
/>
```

Add a fourth item with food that starts with the same letter as your name and show an image of it in the list. 320x362 is recommended for the size. Show screenshots of both the list and detail screens after making the change. They should look like the following but with your name.


11:47
index
Fruit List
Apple
Tap for details
Banana
Tap for details
Cherry
Tap for details
Pineapple
Tap for details
11:48
detail
Pineapple (ID: 4)
Pat's perfect pineapple.